

第五讲：预期——转移动态

异质性主体宏观经济学的计算方法

孙振越 (Jeffrey Sun)

2026 年 5 月 15 日

多伦多大学

HouseholdStages

HouseholdStages

- 用于构建异质性主体宏观模型家庭模块的阶段（Stages）实现。
- Auclert–Bardóczy–Rognlie–Straub 的 **SSJ 工具包**（Python）也有阶段实现。
- Econ-ARK 也提供了一种形式的阶段。

这个包提供了什么

- **阶段实现。** MarkovStage、WealthChangeStage、ConsumptionSavingsStage、LogitChoiceStage、MigrationStage、BorrowingConstraintStage、AssetPriceChangeStage ...
- **组合。** ○ 串接阶段；lift_moments 附加积分。
- **内层求解辅助函数。** 给定 env（价格与参数），计算稳态与转移的 V 、 Λ 。
- **函子式提升。** lift_jacobian（自动微分）、lift_gpu。
- **SSJ 工具。** 序列空间雅可比（在 L6+ 中讲解）。

这个包不做什么

这个库处理**给定 env 下的家庭模块**。它不：

- 决定你的总量状态。
- 从 $\bar{K}$ 计算价格（厂商模块）。
- 为校准或市场出清运行外层循环。

为什么这样拆分

- 家庭模块：跨异质性主体模型的**通用组件**。
- 均衡、校准、转移：**模型特定**。

稳态内层求解辅助函数

```
# 在固定的 env 下， $v$  反向迭代至不动点。
solve_vfi_steady_state_given_env!(hh, env)

# 用反向扫描填充的策略，将  $\lambda$  正向迭代至平稳。
solve_lambda_steady_state_given_env!(hh)

# 两者的打包：先  $v$  后  $\lambda$ ，再附加矩。
solve_steady_state_given_env!(hh, env)
```

三者都将 env **视为给定**。它们迭代家庭模块，但不更新价格或参数。

以阶段构建 Aiyagari 稳态

与之前相同的模型

- 阶段链（时间顺序）：

$z_shock \circ income \circ savings.$

- 厂商。柯布—道格拉斯，劳动 L 固定。
- 均衡。 $R = R^{\text{implied}}(K^{\text{supplied}}(R))$ 。
- 外层循环。对 R 进行带阻尼的试错调价（tatonnement）。

状态布局

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(0, 100; length=400, spacing=:log)),  
    StateAxis(:z,      [0.6, 1.0, 1.4]),  
)
```

- **wealth** (连续, 400 点对数网格)。
- **z** (生产率; 离散, 3 个水平)。
- 闭包读取 `cell.wealth`、`cell.z`。

阶段 1: z 冲击

```
z_shock = MarkovStage(layout; axis=:z, transition=p.Π_z)
```

阶段 2：收入

```
income = WealthChangeStage(layout; wealth_post=(cell; env) -> env.R * cell.wealth + env.w *  
    cell.z)
```

- 之前的例子是对齐到网格。库函数则连续地重新插值。

阶段 3：消费储蓄

```
savings = ConsumptionSavingsStage(layout;  $\beta$ =p. $\beta$ , utility=(cell, c; env) -> u_crra(c, Val(p. $\sigma$ )),  
    monotone_search=:divide_conquer)
```

- :divide_conquer: 当策略关于 b 单调时, 采用更快的 $O(N \log N)$ 算法。

组合

```
hh = z_shock ◦ income ◦ savings
```

- ○ 按时间顺序组合各阶段。
- 家庭模块本身就是一个阶段：拥有自己的 backward! 和 forward!。

定义矩

```
hh = define_moments!(hh; K_supplied=at_end(integrand=:wealth, reduce=sum))
```

- `define_moments!`: 在 Λ 上定义积分。这里 $K^{\text{supplied}} = \int b d\Lambda^{\text{end}}$ 。

给定 R 求解稳态

```
(;K, w) = aiyagari_supply_side(R, p)
env = make_env(hh; R, w)
res = solve_steady_state_given_env!(hh, env)
K_supplied = res.moments.K_supplied
```

- 第 1 行。由 R 得到厂商侧的 K 和工资 w 。
- 第 2 行。构建 env: (R, w) 。
- 第 3 行。求出稳态 V 和 Λ 。
- 第 4 行。读出 K^{supplied} 。

外层试错调价循环

```
R = R_init
while err >= rtol
    env = make_env(hh; R, w=aiyagari_supply_side(R, p).w)
    (;V, Λ, moments) = solve_steady_state_given_env!(hh, env)
    R_implied = aiyagari_prices(moments.K_supplied, p).R
    err = abs(R_implied - R)
    R += update_speed*(R_implied - R)           # 带阻尼的试错调价
end
```

每一轮： $R \rightarrow (K, w) \rightarrow V, \Lambda \rightarrow K^{\text{supplied}} \rightarrow R^{\text{implied}}$ ，然后更新 R 。

当 $|R^{\text{implied}} - R| < \text{rtol}$ 时停止。

对价格进行试错调价是通用做法：家庭模块是价格接受者，外层循环搜索使市场出清的价格。对 $\bar{K}$ 进行试错调价在 Aiyagari 中恰好可行，因为资产市场与商品市场出清重合，但这并不具有一般性。

比较静态

比较静态

改变一个参数，重新求解稳态。

可以把它看作对冲击的**长期**反应。

在示例校准中：

- $A = 1 \rightarrow \bar{K}_{\text{pre}} = [\text{verify}], \bar{R}_{\text{pre}} = [\text{verify}].$
- $A = 1.05 \rightarrow \bar{K}_{\text{post}} = [\text{verify}], \bar{R}_{\text{post}} = [\text{verify}].$

比较静态

比较静态告诉你对冲击的**长期**反应。

它不告诉你：

- 转移的速度
- 转移的路径
- 冲击对当时在世者的福利影响。

要回答这些，我们需要转移动态。

转移动态

转移动态

- **稳态。**单一的 (K, V, Λ) 。不随时间变化。
- **转移。**一个序列 $\{(K_t, V_t, \Lambda_t)\}_{t=1, \dots, T}$ 。经济正在运动。

MIT 冲击

在 $t = 1$ 处的一次**永久、未预期到**的冲击。

- $t = 0$: 处于稳态。
- $t = 1$: 参数永久改变。主体感到意外。
- $t \geq 1$: 对外生与内生变量的整条未来路径具有完美预见。

示例：TFP 冲击

在 $t = 1$ 处的一次永久正向 TFP 冲击：

$$A_t = 1.05 \quad \text{for all } t \geq 1, \quad A_0 = 1.$$

- 唯一的外生变化。
- $\{R_t, w_t, K_t, V_t, \Lambda_t\}$ 作为**反应**而改变。
- 设 $T = 100$ ，并令 $V_{T+1} = V^{\text{new_steady_state}}$ 以闭合模型。

求解 MIT 转移

问题陈述

- 给定: $\{A_t\}_{t=1}^T$ 。
- 求: $\{R_t, V_t, \Lambda_t\}$, 使得
 - $V_{T+1} = V_{\text{ss}^{\text{post}}}$ (终端)。
 - $\Lambda_1 = \Lambda_{\text{ss}^{\text{pre}}}$ (初始)。
 - $V_t = \text{backward}(V_{t+1}, \text{env}_t)$.
 - $\Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1}, \text{env}_t)$.
 - 市场出清: $R_t = R^{\text{implied}}(K_t^{\text{supplied}})$ 。

算法

1. 猜测路径 $\{R_t\}$ 。
2. 将 V 反向迭代。

$$V_{T+1} \leftarrow V_{\text{SS}^{\text{Post}}}, \quad V_t = \text{backward}(V_{t+1}, \text{env}_t).$$

3. 将 Λ 正向模拟。

$$\Lambda_1 \leftarrow \Lambda_{\text{SS}^{\text{Pre}}}, \quad \Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1}, \text{env}_t).$$

4. 更新 R_{path} 。

$$R_t \leftarrow (1 - s) R_t + s R_t^{\text{implied}}, \quad R_t^{\text{implied}} = R^{\text{implied}}(K_t^{\text{supplied}}).$$

注记：求根

归根结底，问题是在序列空间中寻找超额需求函数的根：

$$\text{excess_demand}(\{R_t\}) = \vec{0}.$$

为演示起见，我们使用简单而稳健的试错调价方法。

使用序列空间雅可比的基于梯度的方法可以快得多。

算法

```
hh_path = [aiyagari_household(p_new) for _ in 1:T] # 每期一条链
R_path = collect(range(pre.R, post.R; length=T)) # 初始猜测
V_path[T+1] .= post.V;  $\Lambda$ _path[1] .= pre. $\Lambda$  # 边界条件
while err >= rtol
    env_path = [make_env(hh_path[t]; R=R_path[t], w=aiyagari_supply_side(R_path[t], p_new).w) for t in 1:T]
    for t in T:-1:1 # 反向扫描
        copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_path[t]))
    end
    for t in 1:T # 正向扫描
        copyto!( $\Lambda$ _path[t+1], forward!(hh_path[t],  $\Lambda$ _path[t]))
        K_S = compute_moments(hh_path[t],  $\Lambda$ _path[t+1], env_path[t]).K_supplied
        R_implied_path[t] = aiyagari_prices(K_S, p_new).R
    end
    err = maximum(abs.(R_implied_path .- R_path))
    R_path .+= update_speed .* (R_implied_path .- R_path) # 带阻尼的试错调价
end
```

实现

初始稳态

两者我们都需要：

- V_{ss}^{post} ：反向扫描的终端条件。
- Λ_{ss}^{pre} ：正向扫描的初始条件。
- 第 2 节的试错调价能给出两者（在两端各运行一次）。
- **设置：** $V_{ss}^{post} \rightarrow V_{path}[T+1]$ ； $\Lambda_{ss}^{pre} \rightarrow \Lambda_{path}[1]$ ；初始 $\{R_t\}$ 取从 R_{ss}^{pre} 到 R_{ss}^{post} 的一条直线。

反向扫描

```
for t in T:-1:1
    env_t = make_env(hh_path[t]; R=R_path[t], w=aiyagari_supply_side(R_path[t], p_new).w)
    copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_t))
end
```


正向扫描（重新安置核!）

```
for t in 1:T
    env_t = make_env(hh_path[t]; R=R_path[t], w=aiyagari_supply_side(R_path[t], p_new).w)
    copyto!( $\Lambda$ _path[t+1], forward!(hh_path[t],  $\Lambda$ _path[t]))
    K_S = compute_moments(hh_path[t],  $\Lambda$ _path[t+1], env_t).K_supplied
    R_implied_path[t] = aiyagari_prices(K_S, p_new).R
end
```

外层试错调价更新

```
err = maximum(abs.(R_implied_path .- R_path))  
R_path .+= update_speed .* (R_implied_path .- R_path)
```

脉冲响应：资本

形状。

- ΔA 抬高 R 。
- 更高的 $R \rightarrow$ 更多储蓄。
- 更多储蓄 $\rightarrow$ 更高的 $K \rightarrow$ 更低的 R 。具有自我纠正性。

滞后。 财富分布具有惯性。 K 的峰值滞后于 A 的峰值约 ~ 5 期。

脉冲响应： R_t 与 w_t

R_{to}

- 在冲击当期跳升（ A 在 K 变动之前先上升）。
- 随着 K 累积，衰减回 $R_{ss}^{post} = 1/\beta$ 。
- 在 Aiyagari 中 $R_{ss}^{pre} = R_{ss}^{post}$ ； K 吸收了这一永久性移动。

w_{to}

- 形态相同，但冲击当期符号相反。
- 随 K 上升，达到一个永久更高的 w_{ss}^{post} 。

福利

- 富裕主体：在 R 上获得短暂的资本利得。
- 依赖工资的主体：受益于 w 一侧。

对 Aiyagari，我们逐期得到一个福利效应的分布。厘清这两条渠道正是 MIT 冲击让你能够做到的一件事。

把 T 取得足够大

- $T = \bar{K}$ 与 V 已收敛到冲击后稳态的时间跨度，使得 $V_{T+1} \approx V_{ss}^{\text{post}}$ 。
- 太小 $\rightarrow$ 终端条件会污染整条路径。

经验法则。取使得 $|\bar{K}_T - \bar{K}_{ss}^{\text{post}}| < 10^{-3}$ 的 T 。我们使用 $T = 200$ 。